

AGV DOCUMENTATION

TASK – 1

Subtask -1

Lucas-Kanade sparse optical flow estimation

SOURCES:

1. Article (<https://nanonets.com/blog/optical-flow/>)
2. Youtube → **Optical Flow | Structure from Motion** (<https://www.youtube.com/playlist?list=PL2zRqk16wsdoYzrWStffqBAoUY8XdvatV>)
3. **OpenCV Python Course for Beginners** (https://www.youtube.com/watch?v=MVkny_XLK_U)
4. **OpenCV Course—Full Tutorial with Python** (<https://www.youtube.com/watch?v=oXlwWbU8I2o>)
5. CHATGPT

TASK – 5

SOURCES:

1. Research Paper
(<https://www.cambridge.org/core/journals/robotica/article/2dlidarbased-localization-method-for-indoor-mobile-robots-using-correlative-scan-matching/291583763D866B1739AEF58ADC34D659>)

2. CHATGPT

STEP BY STEP CODE:

1. The preprocessed map is given, which is read through OpenCV, and the image is converted to the HSV colour space .

Then, these green pixels are masked to make them white [255, 255, 255] and converted to **grayscale** since scan matching works best with a single-channel image.

```
def preprocess_map(map_path):  
    """Loads and processes the given map by converting green pixels to white."""  
    map_img = cv2.imread(map_path) # Load the map image  
  
    if map_img is None:  
        raise FileNotFoundError(f"Could not open or find the image at {map_path}")  
  
    # Convert to HSV and replace green with white  
    hsv = cv2.cvtColor(map_img, cv2.COLOR_BGR2HSV)  
    lower_green = np.array([35, 100, 100])  
    upper_green = np.array([85, 255, 255])  
    mask = cv2.inRange(hsv, lower_green, upper_green)  
    map_img[mask > 0] = [255, 255, 255] # Replace green pixels with white  
  
    # Convert to grayscale  
    gray_map = cv2.cvtColor(map_img, cv2.COLOR_BGR2GRAY)  
  
    return gray_map
```

2. Loading Lidar data:

I read the LiDAR data from the text file, and the code is written to ignore the lines having headers and irrelevant information.

(x,y) coordinates of LiDAR data returned as a NumPy array of valid (x,y) points.

```
def load_lidar_data(lidar_file):
    """Loads LiDAR scan data, extracting (x, y) positions while skipping metadata."""
    valid_points = []

    with open(lidar_file, "r") as file:
        for line in file:
            line = line.strip()

            if line.startswith("#", "PARAM", "SYNC", "ODOM"):
                continue

            parts = line.split()
            if parts[0] in {"FLASER", "RLASER"}:
                try:
                    num_readings = int(parts[1])
                    x, y = map(float, parts[num_readings+2:num_readings+4])
                    valid_points.append([x, y])
                except (ValueError, IndexError):
                    print(f"Skipping invalid LiDAR line: {line}")

    if not valid_points:
        raise ValueError(f"Error: No valid LiDAR data found in {lidar_file}")

    lidar_data = np.array(valid_points)
    print(f"Loaded LiDAR data shape: {lidar_data.shape}")
    return lidar_data
```

This is stored as a 7374×2 matrix, having x and y coordinates as the 2 column entries in the the matrix.

```
Loaded LiDAR data shape: (7374, 2)
[[-6.02000e-04  1.27270e-02]
 [-6.02000e-04  1.27270e-02]
 [-6.02000e-04  1.27270e-02]
 ...
 [-1.78057e+01 -3.18096e+00]
 [-1.78057e+01 -3.18096e+00]
 [-1.78057e+01 -3.18096e+00]]
```

3. Correlative scan matching:

map_img → 2D map of the environment

lidar_scan → Set of 2D points (x,y)

search_range → The number of pixels to shift the scan in both x and y directions for finding the best alignment

```
def correlative_scan_matching(map_img, lidar_scan, search_range=10):
    """Performs Correlative Scan Matching (CSM) to estimate position."""
    if lidar_scan.ndim == 1:
        lidar_scan = lidar_scan.reshape(-1, 2) # Ensure shape is (N,2)

    best_match = None
    best_score = -np.inf
    best_x, best_y = 0, 0

    for dx in range(-search_range, search_range):
        for dy in range(-search_range, search_range):
            transformed_scan = lidar_scan + np.array([dx, dy]) # Use NumPy array
            score = compute_match_score(map_img, transformed_scan)

            if score > best_score:
                best_score = score
                best_match = transformed_scan
                best_x, best_y = dx, dy

    return best_x, best_y, best_match
```

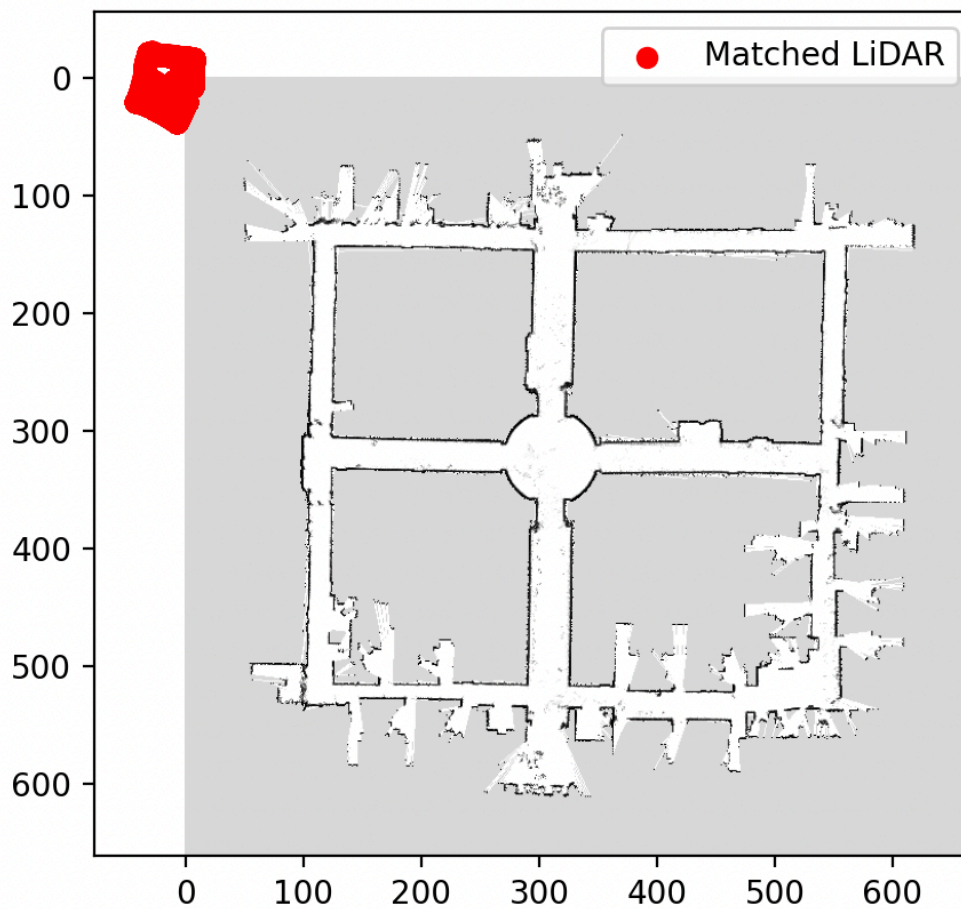
First, it is checked if its dimension is 2; if not, it is made in 2 dimensions.

Variables are next initialised

Search is then done to find the best match.

4. We calculate the score.

FINALLY, THE IMAGE I GOT WAS VERY DISTORTED:



TASK – 6

Sparse Reconstruction:

Following the instructions given in section 2.5

1. Load the two temple images and the points from `data/some_corresp.npz`:

How to load images in OpenCV⇒

Source – OpenCV **Course – Full Tutorial with Python**

(<https://www.youtube.com/watch?v=oXlwWbU8l2o>)

Youtube video on Computer Vision (<https://youtube.com/playlist?list=PL2zRqk16wsdoCCLpou-dGo7QQNks1Ppzo&si=CTTwLwlo2ox1vpR8>)

I loaded all the images and data and called the function `eight_point`

```
data = np.load("data/intrinsics.npz")
K1, K2 = data["K1"], data["K2"]

im1 = cv2.imread("data/im1.png", cv2.IMREAD_GRAYSCALE)
im2 = cv2.imread("data/im2.png", cv2.IMREAD_GRAYSCALE)

M = max(im1.shape)
data_pts = np.load("data/some_corresp.npz")
pts1, pts2 = data_pts["pts1"], data_pts["pts2"]

F = eight_point(pts1, pts2, M)
```

VS Code of

Arrays in the .npz file:

['K1', 'K2']

Array K1:

```
[[1.5204e+03  0.0000e+00  3.0232e+02]
 [0.0000e+00  1.5259e+03  2.4687e+02]
 [0.0000e+00  0.0000e+00  1.0000e+00]]
```

Array K2:

```
[[1.5204e+03  0.0000e+00  3.0232e+02]
 [0.0000e+00  1.5259e+03  2.4687e+02]
 [0.0000e+00  0.0000e+00  1.0000e+00]]
```

Intrinsic Matrix

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

Calibration Matrix:

$$K = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

Intrinsic Matrix:

$$M_{int} = [K|0] = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Upper Right Triangular Matrix

$$\tilde{\mathbf{u}} = [K|0] \tilde{\mathbf{x}}_c = M_{int} \tilde{\mathbf{x}}_c$$

We can see that the intrinsic file matches the format it is expected to be in (Position of zeroes and 1)

2. Run `eight_point` to compute the fundamental matrix F

```
def normalize_points(pts, M):  
    T = np.array([[1/M, 0, 0], [0, 1/M, 0], [0, 0, 1]])  
    pts_hom = np.column_stack((pts, np.ones(len(pts))))  
    pts_norm = (T @ pts_hom.T).T[:, :2]  
    return pts_norm, T
```

This function takes in an $N \times 2$ matrix and returns a different scaled-down version of the matrix in $N \times 2$ form.

The function `np.column_stack` adds a column matrix filled with ones ($N \times 1$) to the transposed version of our input matrix.

- If `pts` is:

$$\begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ x_3 & y_3 \end{bmatrix}$$

It becomes:

$$\begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{bmatrix}$$

Then we perform matrix multiplication of matrix T and transpose our newly made matrix.

Taking the transpose again of the matrix and eliminating the last row to obtain a scaled-down version of our matrix

- This multiplies each point by the transformation matrix **T** using matrix multiplication.

$$\begin{bmatrix} \frac{1}{M} & 0 & 0 \\ 0 & \frac{1}{M} & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{bmatrix}^T$$

This results in:

$$\begin{bmatrix} \frac{x_1}{M} & \frac{x_2}{M} & \frac{x_3}{M} \\ \frac{y_1}{M} & \frac{y_2}{M} & \frac{y_3}{M} \\ 1 & 1 & 1 \end{bmatrix}$$

- **.T** transposes it back to $(N, 3)$:

$$\begin{bmatrix} \frac{x_1}{M} & \frac{y_1}{M} & 1 \\ \frac{x_2}{M} & \frac{y_2}{M} & 1 \\ \frac{x_3}{M} & \frac{y_3}{M} & 1 \end{bmatrix}$$

- **[:, :2]** removes the last column (homogeneous coordinate), leaving:

$$\begin{bmatrix} \frac{x_1}{M} & \frac{y_1}{M} \\ \frac{x_2}{M} & \frac{y_2}{M} \\ \frac{x_3}{M} & \frac{y_3}{M} \end{bmatrix}$$

```

def eight_point(pts1, pts2, M):
    pts1_norm, T1 = normalize_points(pts1, M)
    pts2_norm, T2 = normalize_points(pts2, M)

    A = np.column_stack([
        pts1_norm[:, 0] * pts2_norm[:, 0],
        pts1_norm[:, 0] * pts2_norm[:, 1],
        pts1_norm[:, 0],
        pts1_norm[:, 1] * pts2_norm[:, 0],
        pts1_norm[:, 1] * pts2_norm[:, 1],
        pts1_norm[:, 1],
        pts2_norm[:, 0],
        pts2_norm[:, 1],
        np.ones(len(pts1))
    ])

    _, _, Vt = np.linalg.svd(A)
    F = Vt[-1].reshape(3, 3)

    U, S, Vt = np.linalg.svd(F)
    S[-1] = 0 # Enforce rank 2 constraint
    F_refined = U @ np.diag(S) @ Vt

    F_final = T2.T @ F_refined @ T1
    return F_final

```

V_t is a (9 X 9) matrix whose last row is taken and the 9 entries are converted to 3 X 3 matrix to form matrix F (Fundamental matrix).

SVD (Single Value Decomposition) decomposes any matrix into 3 parts :

- **U**: Left singular vectors (orthogonal matrix)
- **S**: Singular values (diagonal matrix)
- **V_t** : Right singular vectors (orthogonal matrix)

Here, the solution for F in $AF = 0$ is given by the last column of V in $A = USV_t$, which is the last row of V_t .

' _ ' $\Rightarrow$ Means that value is ignored, i.e., U and S ignored

Now SVD (Single Value Decomposition) of F is computed, and since S is a diagonal (3x3) matrix, it contains only 3 diagonal values, of which the last entry is made 0 to enforce its rank = 2.

Now, F_{refined} is reconstructed using a new value of S .

F_{final} is the conversion of F_{refined} , which is computed in the normalised system, back to our original system.

- You must enforce the rank 2 constraint on F before unscaling. Recall that a valid fundamental matrix F will have all epipolar lines intersect at a certain point, meaning that there exists a non-trivial null space for F . In general, with real points, the eight-point solution for F will not come with this condition. To enforce the rank 2 constraint, decompose F with SVD to get the three matrices U , Σ , V such that $F = U\Sigma V^T$. Then force the matrix to be rank 2 by setting the smallest singular value in Σ to zero, giving you a new Σ' . Now compute the proper fundamental matrix with $F' = U\Sigma'V^T$.

The **epipolar constraint** for a fundamental matrix is:

$$\mathbf{x}_2^T F \mathbf{x}_1 = 0$$

Since normalization transforms points as:

$$\mathbf{x}_1^{\text{norm}} = T_1 \mathbf{x}_1$$

$$\mathbf{x}_2^{\text{norm}} = T_2 \mathbf{x}_2$$

The fundamental matrix in **normalized space** satisfies:

$$\mathbf{x}_2^{\text{norm}^T} F_{\text{refined}} \mathbf{x}_1^{\text{norm}} = 0$$

Expanding:

$$(T_2 \mathbf{x}_2)^T F_{\text{refined}} (T_1 \mathbf{x}_1) = 0$$

which simplifies to:

$$\mathbf{x}_2^T (T_2^T F_{\text{refined}} T_1) \mathbf{x}_1 = 0$$

Since the fundamental matrix in the **original coordinates** must satisfy $\mathbf{x}_2^T F \mathbf{x}_1 = 0$, we identify:

$$F_{\text{final}} = T_2^T F_{\text{refined}} T_1$$

```
Fundamental Matrix: [[ 3.56440672e-09 -1.30829066e-07  3.04775126e-05]
 [-5.92131870e-08 -1.31095126e-09 -1.08013400e-03]
 [-1.65029959e-05  1.12471525e-03 -4.16583180e-03]]
```

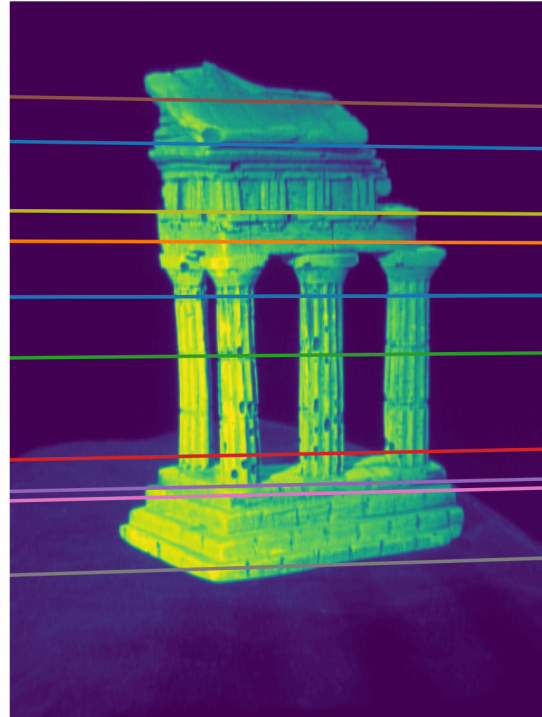
This is the fundamental matrix obtained.

On selecting random points this is the image I am getting.

Select a point in this image



Verify that the corresponding point is on the epipolar line in this image



For this part, the resources used are:

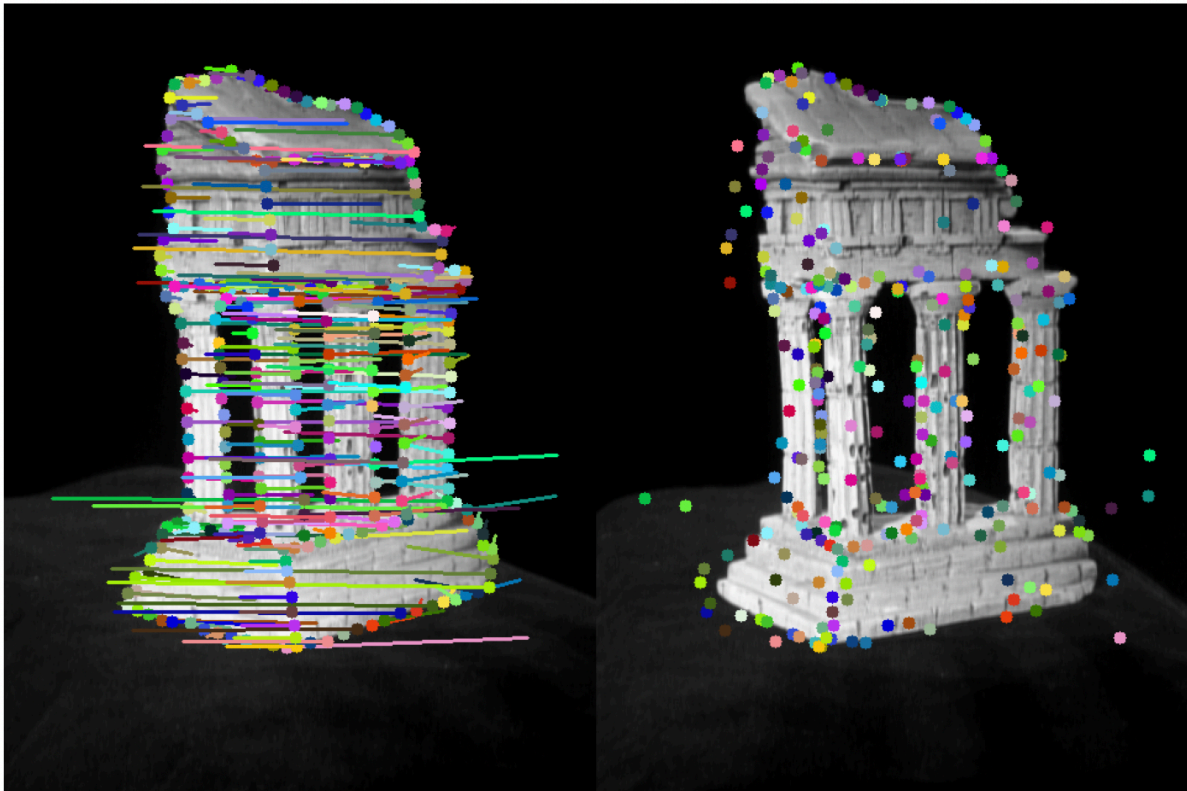
1. ChatGPT
 2. **8 Point Algorithm – 5 Minutes with Cyrill** (<https://www.youtube.com/watch?v=z92eUJjIJeY>)
 3. **Estimating Fundamental Matrix | Uncalibrated Stereo** (<https://www.youtube.com/watch?v=izpYAwJOHlw>)
-
3. Load the points in image 1 contained in data/temple coords.npz, and run your epipolar correspondences on them to get the corresponding points in image 2

```
Arrays in the .npz file:
['pts1']
Array pts1:
[[184  54]
 [172  60]
 [162  62]
 [150  64]
 [138  65]
 [134  76]
 [135  93]
 [132 107]
 [128 118]
 [127 134]
 [132 146]
 [135 157]
 [133 169]
 [137 182]
 [129 191]
 [126 204]
 [127 216]
 [132 226]
 [131 237]
 [138 247]
 [143 259]
 [145 275]
 [144 288]
 [147 300]
```

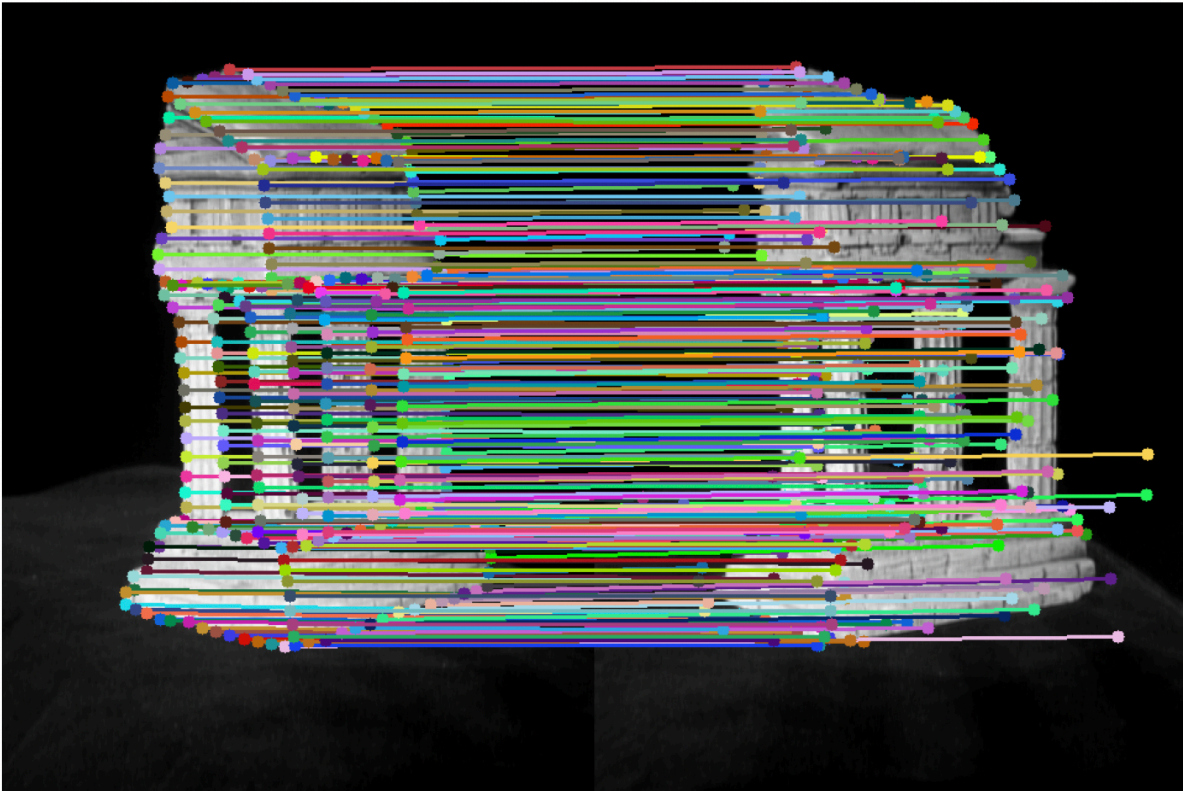
I have loaded the points in this file, and now I need to find the corresponding points in image-2.

When I attempted to run the function `epipolarMatchGUI(I1, I2, F)`, it repeatedly failed and exited. So, I made the correspondence between the two images by writing my function.

Epipolar Correspondences



8 Epipolar Correspondences



First, when I made a correspondence between the 2 images, I did not filter only a few points, due to which I got many correspondences.

But when I tried with this code to get only 8 points of correspondence between the two, it gave unusual results, as shown with the given code.

```

def epipolar_correspondences(im1, im2, F, pts1):
    if best_match:
        pts2.append(best_match)
    pts2 = np.array(pts2)

    # Convert images to color if grayscale
    im1_color = cv2.cvtColor(im1, cv2.COLOR_GRAY2BGR) if len(im1.shape) == 2 else im1.copy()
    im2_color = cv2.cvtColor(im2, cv2.COLOR_GRAY2BGR) if len(im2.shape) == 2 else im2.copy()

    # Create a blank canvas for visualization
    combined_image = np.hstack([im1_color, im2_color])

    num_points = 8
    selected_indices = np.random.choice(len(pts1), num_points, replace=False) # Get 8 unique indices
    selected_pts1 = pts1[selected_indices]

    for (x1, y1), (x2, y2) in zip(selected_pts1, pts2):
        color = tuple(np.random.randint(0, 255, 3).tolist()) # Random color

        # Offset x2 for the second image in the combined view
        x2_offset = x2 + w1

        # Draw circles on both images
        cv2.circle(combined_image, (x1, y1), 5, color, -1)
        cv2.circle(combined_image, (x2_offset, y2), 5, color, -1)

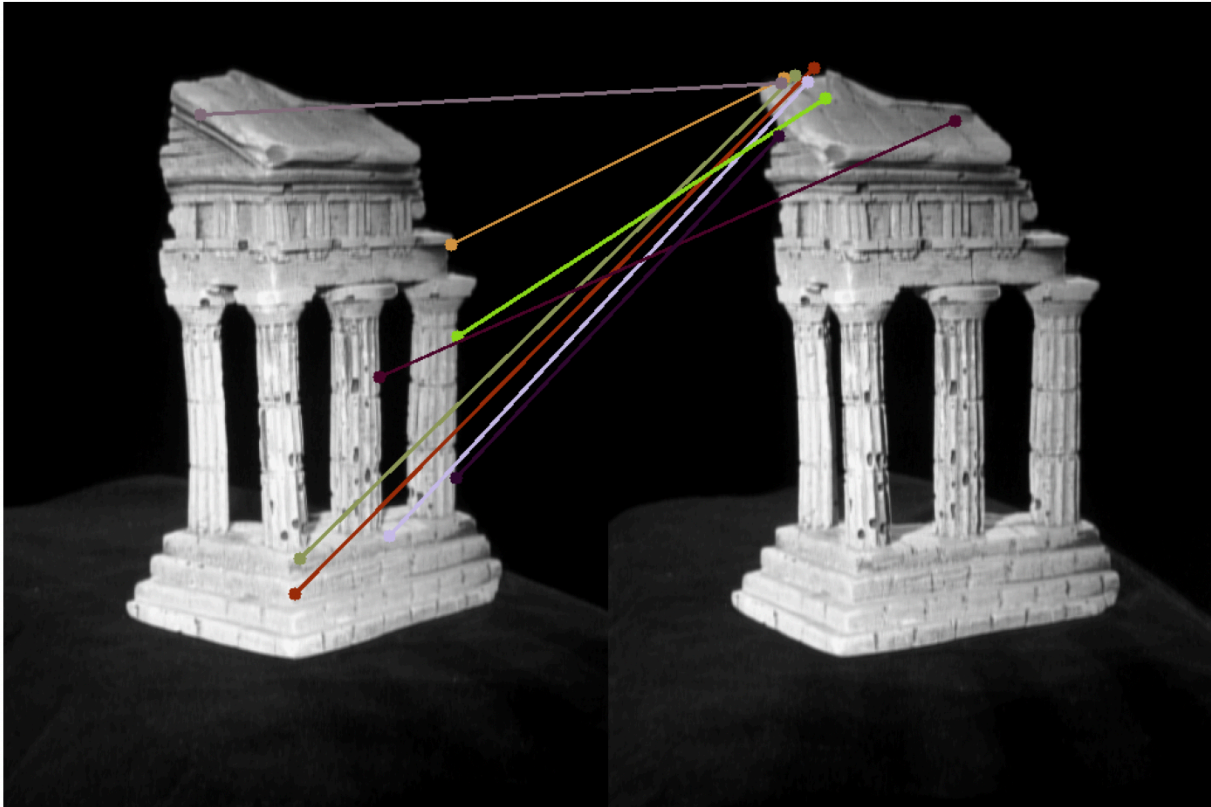
        # Draw lines connecting correspondences
        cv2.line(combined_image, (x1, y1), (x2_offset, y2), color, 2)

    # Show the final image
    plt.figure(figsize=(12, 6))
    plt.imshow(cv2.cvtColor(combined_image, cv2.COLOR_BGR2RGB))
    plt.axis("off")
    plt.title("8 Epipolar Correspondences")
    plt.show()

    return np.array(pts2)

```

8 Epipolar Correspondences



It was because I converted it to a greyscale image to place on a black canvas, to show that when I mapped a point in image 1, it got mapped to a random point in image 2 as it searched over the entire image.

To correct this, I mapped it to the corresponding points in the two images directly. After which, I got the correct mapping through this code.


```

def epipolar_correspondences(im1, im2, F, pts1):

    if best_match:
        pts2.append(best_match)
    pts2 = np.array(pts2)

    # Create a blank canvas for visualization
    combined_image = np.hstack((im1, im2)) # Concatenate images side by side

    # Select 8 random points
    num_points = 8
    selected_indices = np.random.choice(len(pts1), num_points, replace=False)
    selected_pts1 = pts1[selected_indices]
    selected_pts2 = pts2[selected_indices]

    for (x1, y1), (x2, y2) in zip(selected_pts1, selected_pts2):
        color = tuple(np.random.randint(0, 255, 3).tolist()) # Random color

        # Offset x2 for the second image in the combined view
        x2_offset = x2 + w1

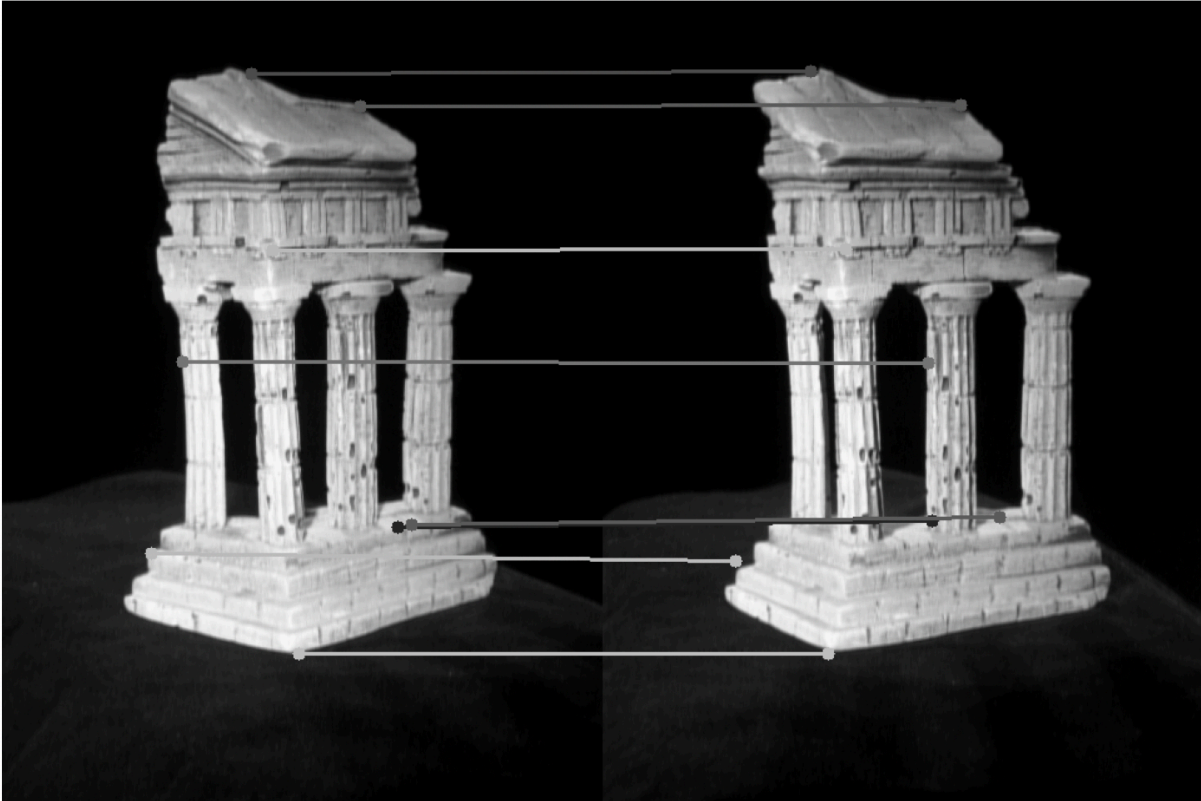
        # Draw circles on both images
        cv2.circle(combined_image, (x1, y1), 5, color, -1)
        cv2.circle(combined_image, (x2_offset, y2), 5, color, -1)

        # Draw lines connecting correspondences
        cv2.line(combined_image, (x1, y1), (x2_offset, y2), color, 2)

    # Show the final image
    plt.figure(figsize=(12, 6))
    plt.imshow(cv2.cvtColor(combined_image, cv2.COLOR_BGR2RGB))
    plt.axis("off")
    plt.title("8 Epipolar Correspondences")
    plt.show()

```

8 Epipolar Correspondences



4. Load data/intrinsics.npz and compute the essential matrix E .

Intrinsics gives the camera parameters for different positions 1 and 2.


```
Arrays in the .npz file:
['K1', 'K2']
Array K1:
[[1.5204e+03  0.0000e+00  3.0232e+02]
 [0.0000e+00  1.5259e+03  2.4687e+02]
 [0.0000e+00  0.0000e+00  1.0000e+00]]
Array K2:
[[1.5204e+03  0.0000e+00  3.0232e+02]
 [0.0000e+00  1.5259e+03  2.4687e+02]
 [0.0000e+00  0.0000e+00  1.0000e+00]]
```

```
Essential Matrix: [[ 8.23954018e-03 -3.03520601e-01 -1.12915154e-03]
 [-1.37373313e-01 -3.05238066e-03 -1.67598594e+00]
 [-4.56779305e-02  1.65535639e+00 -2.87296484e-03]]
```

This is the derivation of how the essential matrix is derived from the fundamental matrix. Direct final relation is used in the function written in code.

Epipolar Constraint

Substituting into the epipolar constraint gives:

$$[x_l \ y_l \ z_l] \left(\begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} + \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \right) = 0$$

$t \times t = 0$

$$[x_l \ y_l \ z_l] \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} = 0$$

Essential Matrix E

Essential Matrix E : Decomposition

$$E = T_{\times} R$$

$$\begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

Given that $T_{\times}$ is a **Skew-Symmetric** matrix ($a_{ij} = -a_{ji}$) and R is an **Orthonormal** matrix, it is possible to "decouple" $T_{\times}$ and R from their product using "Singular Value Decomposition".

How do we find E ?

Relates 3D position (x_l, y_l, z_l) of scene point w.r.t left camera to its 3D position (x_r, y_r, z_r) w.r.t. right camera

$$\mathbf{x}_l^T E \mathbf{x}_r = 0$$

$$\begin{matrix} [x_l & y_l & z_l] & \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} & \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} = 0 \\ \nearrow & \uparrow & \nwarrow \\ \text{3D position in left} & \text{3x3 Essential} & \text{3D position in right} \\ \text{camera coordinates} & \text{Matrix} & \text{camera coordinates} \end{matrix}$$

Fundamental Matrix F

Epipolar constraint:

$$\begin{bmatrix} x_l & y_l & z_l \end{bmatrix} \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} = 0$$

Rewriting in terms of image coordinates:

$$\begin{bmatrix} u_l & v_l & 1 \end{bmatrix} \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

Fundamental Matrix F

$$E = K_l^T F K_r$$

Further functions like `extract_extrinsics` are written, which compute the single value decomposition of the matrix E .

This function computes two possible rotation matrices $R1$ and $R2$ and thus returns 4 pairs of (R, t) i.e. $(R1, t)$, $(R1, -t)$, $(R2, t)$, $(R2, -t)$

The next function, `next_correct_extrinsics`, takes in these 4 (R, t) pairs and also the data from `intrinsic.npz`, which contains the camera parameters.

Further work on the function is in the next steps.

5. Compute the first camera projection matrix $P1$ and use camera 2 to compute the four candidates for $P2$

Here, P , the camera position of the first matrix, is calculated as follows:

In stereo vision or structure from motion, the camera projection matrix P is defined as:

$$P = K[R|t]$$

where:

- K is the **intrinsic matrix** (camera calibration matrix).
- R is the **rotation matrix** describing the camera orientation.
- t is the **translation vector** describing the camera position.

```
P1 = K1 @ np.hstack((np.eye(3), np.zeros((3, 1))))
```

Now we need to compute the 4 candidates for P_2 using the 4 values of (R, t) computed.

This is computed as $P_2 = K_2 [R \mid t]$.

```
P2 = K2 @ np.hstack((R, t.reshape(-1, 1)))
```

6. Run your triangulate function using the four sets of camera matrix candidates, the points from data/temple coords.npz and their computed correspondences.
7. Figure out the correct P_2 and the corresponding 3D points. Hint: You'll get 4 projection matrix candidates for camera2 from the essential matrix. The correct configuration is

the one for which most of the 3D points are in front of both cameras (positive depth).

```
pts3d = triangulate(P1, pts1, P2, pts2)
num_positive_depth = np.sum(pts3d[:, 2] > 0)
if num_positive_depth > best_count:
    best_count = num_positive_depth
    best_R, best_t = R, t
return best_R, best_t
```

The best combination of R and t is found out using the camera point that has the most number of points with positive depth.

```
def triangulate(P1, pts1, P2, pts2):
    pts3d = []
    for i in range(len(pts1)):
        A = np.array([
            pts1[i, 0] * P1[2, :] - P1[0, :],
            pts1[i, 1] * P1[2, :] - P1[1, :],
            pts2[i, 0] * P2[2, :] - P2[0, :],
            pts2[i, 1] * P2[2, :] - P2[1, :]
        ])
        _, _, Vt = np.linalg.svd(A)
        X = Vt[-1]
        X /= X[3]
        pts3d.append(X[:3])
    return np.array(pts3d)
```

In this triangulation function, it is calculated by giving 4 inputs:

1. **P1** → First camera's projection matrix (3×4)
2. **P2** → Second camera's projection matrix (3×4)
3. pts1 → 2D points in **first** image ($N \times 2$)
4. pts2 → 2D points in the **second** image ($N \times 2$)

After calculating A, we will do its single value decomposition and get X
using $AX = 0$, we calculate X

And hence, finally, we get a point in 3-D.

9. Include 3 images of your final reconstruction of the points given in
The file data/temple coords.npz, from different angles, as shown in Figure 6.

Now I have written the code for plotting this 3-D plot of points from 6 different angles in multiples of 30 degrees.

And saved the images.

```

def plot_3d_points(pts3d):
    fig = plt.figure(figsize=(10, 10))
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(pts3d[:, 0], pts3d[:, 1], pts3d[:, 2], marker='o', s=1)
    ax.set_xlabel('X -> horizontal spatial position')
    ax.set_ylabel('Y -> vertical spatial position')
    ax.set_zlabel('Z -> depth')
    ax.set_title('3D Reconstruction')

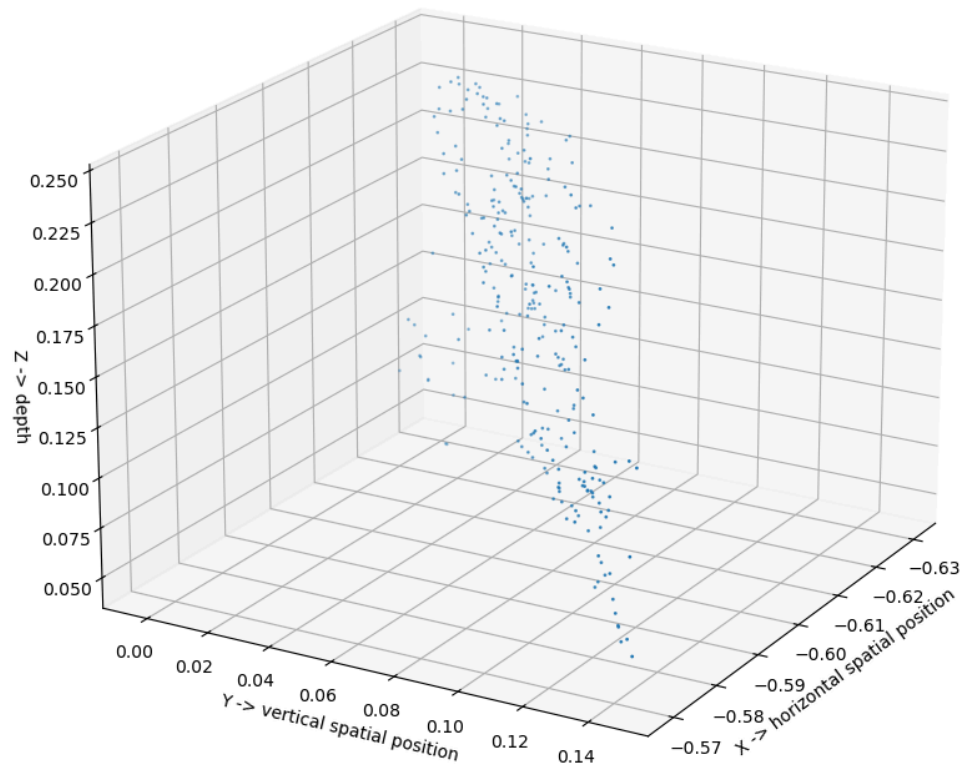
    angles = [30, 60, 90, 120, 150, 180]
    for angle in angles:
        ax.view_init(elev=20, azim=angle)
        plt.savefig(f"3d_reconstruction_{angle}.png")

    plt.show()

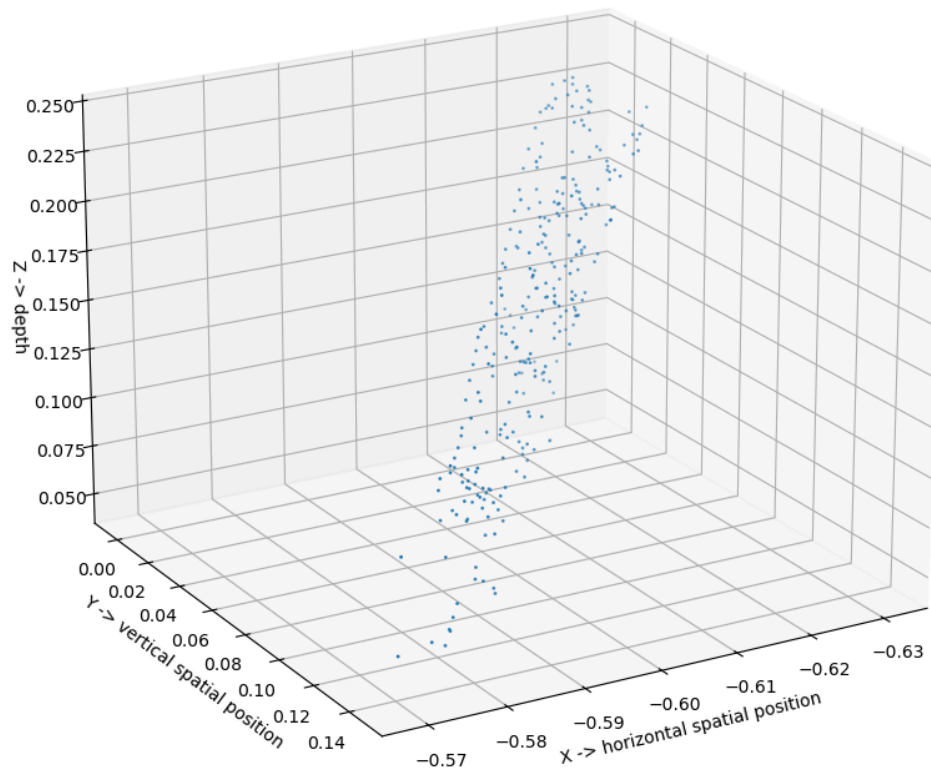
```

angles = [30, 60, 90, 120, 150, 180], respectively.

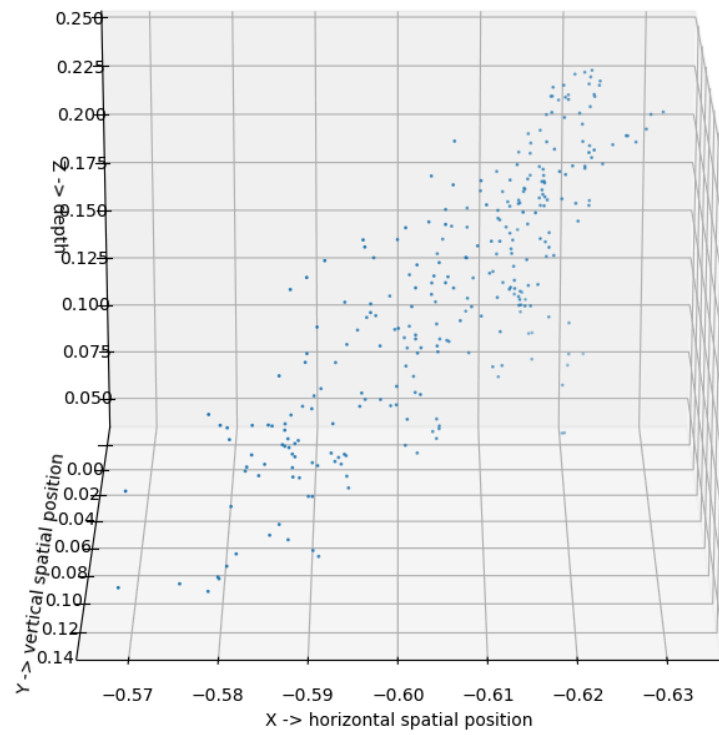
3D Reconstruction



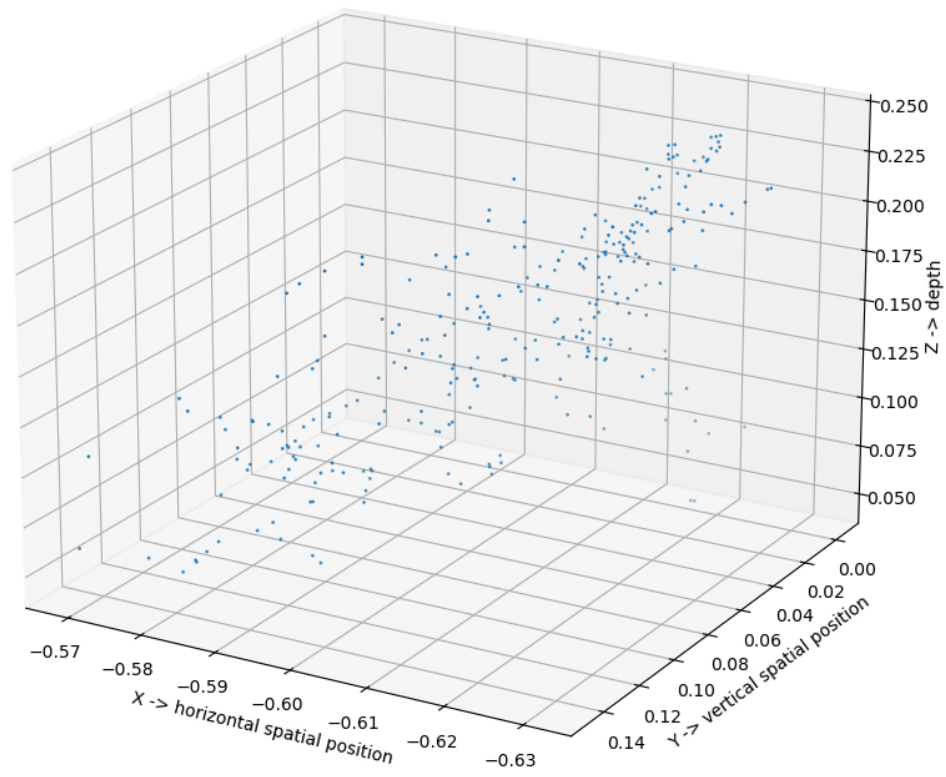
3D Reconstruction



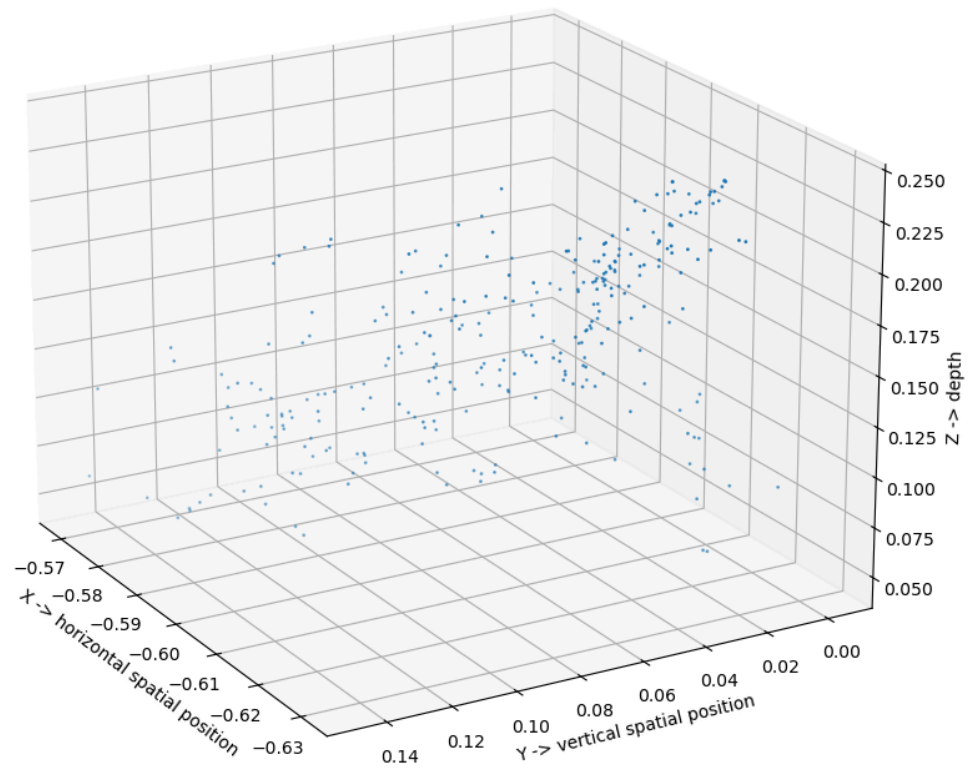
3D Reconstruction



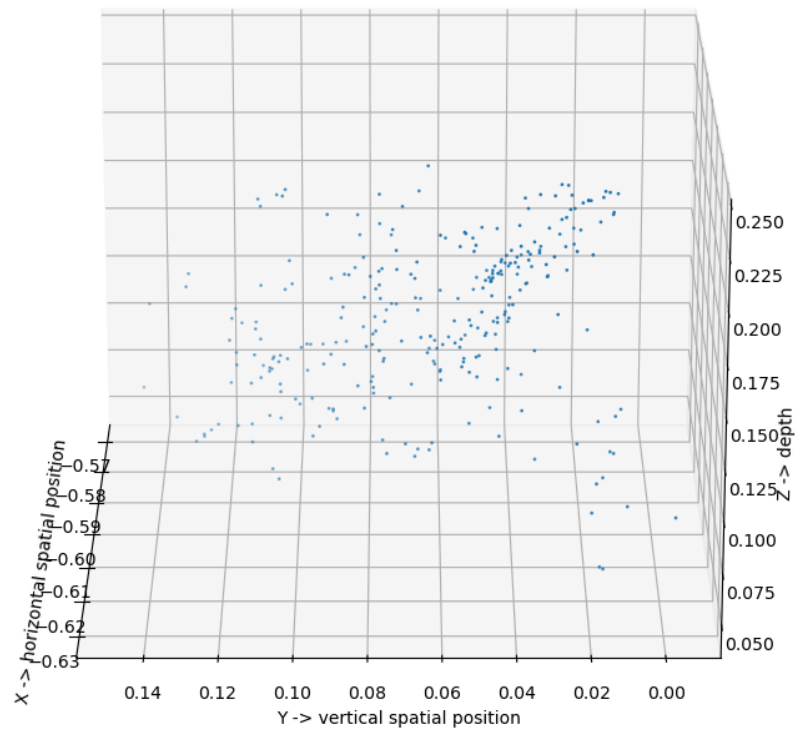
3D Reconstruction



3D Reconstruction



3D Reconstruction



NOTE: I have commented on some parts, as it took the code too long to work; we can uncomment and see the part working.

Stereo Vision and Depth Estimation (SUBTASK-2)

1. Stereo Rectification (rectify_pair function):

```
def rectify_pair(K1, K2, R1, R2, t1, t2):
    # Compute optical centers
    c1 = -np.linalg.inv(K1 @ R1) @ (K1 @ t1)
    c2 = -np.linalg.inv(K2 @ R2) @ (K2 @ t2)

    # Compute the new rotation matrix
    r1 = (c1 - c2) / np.linalg.norm(c1 - c2) # Baseline direction
    r2 = np.cross(R1[2, :], r1) # Orthonormal y-axis
    r2 /= np.linalg.norm(r2)
    r3 = np.cross(r1, r2) # Orthonormal z-axis
    R_new = np.vstack((r1, r2, r3))

    # Compute new camera parameters
    R1p = R_new
    R2p = R_new
    K1p = K1
    K2p = K2
    t1p = -R_new @ c1
    t2p = -R_new @ c2

    # Compute rectification matrices
    M1 = (K1p @ R1p) @ np.linalg.inv(K1 @ R1)
    M2 = (K2p @ R2p) @ np.linalg.inv(K2 @ R2)

    return M1, M2, K1p, K2p, R1p, R2p, t1p, t2p
```

Here:

k1, k2: intrinsic matrices of the two cameras.

R1, R2 : Rotation matrices.

t_1, t_2 : Translation vectors.

Compute the optical centre using: $C = -(K \cdot R)^{\text{inverse}} \cdot (K \cdot t)$

New rotation matrix and camera parameter are computed.

Compute Rectification matrices.

2. Disparity Map (get_disparity function):

Input Taken:

im1, im2: Rectified greyscale images.

max_disp: Maximum disparity (shift) to consider.

win_size: Window size for block matching.


```

def get_disparity(im1, im2, max_disp, win_size):
    h = im1.shape[0]
    w = im1.shape[1]
    disparity_map = np.zeros((h, w))
    half_win = win_size // 2

    for y in range(half_win, h - half_win):
        for x in range(half_win, w - half_win):
            best_offset = 0
            best_score = float('inf')

            for d in range(min(max_disp, x)):
                left_patch = im1[y-half_win:y+half_win+1, x-half_win:x+half_win+1]
                right_patch = im2[y-half_win:y+half_win+1, x-half_win-d:x+half_win+1-d]

                if right_patch.shape != left_patch.shape:
                    continue

                score = np.sum((left_patch - right_patch) ** 2)
                if score < best_score:
                    best_score = score
                    best_offset = d

            disparity_map[y, x] = best_offset

    return disparity_map

```

Iterating over pixels and finding best disparity

3. Depth Map (get_depth function)

```

def get_depth(disparity_map, K1, K2, R1, R2, t1, t2):
    baseline = np.linalg.norm((-np.linalg.inv(K1 @ R1) @ (K1 @ t1) - (-np.linalg.inv(K2 @ R2) @ (K2 @ t2))))
    focal_length = K1[0, 0]

    depth_map = np.zeros_like(disparity_map)
    valid_disp = disparity_map > 0
    depth_map[valid_disp] = (baseline * focal_length) / disparity_map[valid_disp]

    return depth_map

```

It computes baseline → Distance between the cameras

Extract the focal length and compute depth.

4. Draw Epipolar Coordinates (draw_epipolar_lines function)

```
def draw_epipolar_lines(image, num_lines=10):  
    h, w = image.shape  
    step = h // num_lines  
    for i in range(0, h, step):  
        cv2.line(image, (0, i), (w, i), (255, 0, 0), 1)  
    return image  
  
# Load rectified images and draw epipolar lines  
rectified_im1 = draw_epipolar_lines(im1.copy())  
rectified_im2 = draw_epipolar_lines(im2.copy())  
  
# Display rectified images with epipolar lines  
plt.figure(figsize=(10, 5))  
plt.subplot(1, 2, 1)  
plt.imshow(rectified_im1, cmap="gray")  
plt.title("Rectified Image 1 with Epipolar Lines")  
  
plt.subplot(1, 2, 2)  
plt.imshow(rectified_im2, cmap="gray")  
plt.title("Rectified Image 2 with Epipolar Lines")  
  
plt.show()
```

FINAL IMAGES ARE:

